

71. Les collisions

Regarder **AVANT** de bouger, et non reculer après.

1. Ce qu'il faut comprendre

`lire(colonne, ligne)` rend le numéro de la tuile affichée. Comparer à `MUR` suffit : **le nom d'une tuile EST son numéro**.

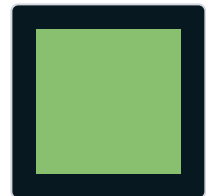
La forme compte. On calcule la position **voulue**, on regarde ce qu'il y a, et l'on n'y va que si c'est libre. Bouger puis reculer marche aussi, mais laisse une image où le personnage est dans le mur — et cette image finit toujours par se voir.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1665 octets** de cartouche.

1. UN DESSIN — MUR

```
Tuile MUR = {
    "#####", "#-----#", "#-----#", "#-----#",
    "#-----#", "#-----#", "#-----#", "#####",
};
```



à l'écran

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    for (uint8_t y = 0; y < 18; y++) { poser(0, y, MUR); poser(19, y, MUR); }
    for (uint8_t x = 0; x < 20; x++) { poser(x, 0, MUR); poser(x, 17, MUR); }
    poser(10, 8, MUR);

    uint8_t c = 5;
    uint8_t l = 8;

    while (true) {
        uint8_t vc = c;
        uint8_t vl = l;

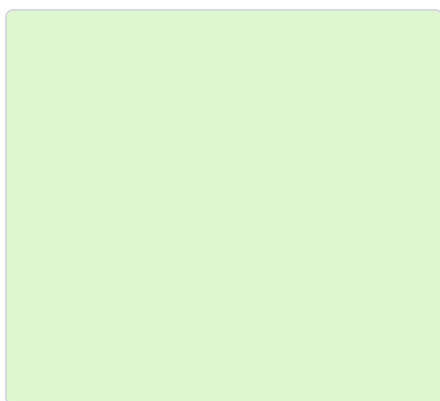
        if (bouton(DROITE)) vc++;
        if (bouton(GAUCHE)) vc--;
        if (bouton(BAS))    vl++;
        if (bouton(HAUT))  vl--;

        /* On regarde AVANT de bouger : la case visée est-elle libre ? */
        if (lire(vc, vl) != MUR) { c = vc; l = vl; }

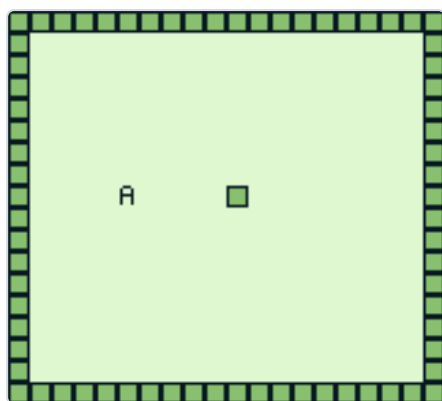
        sprite(0, c * 8, l * 8, 1);
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



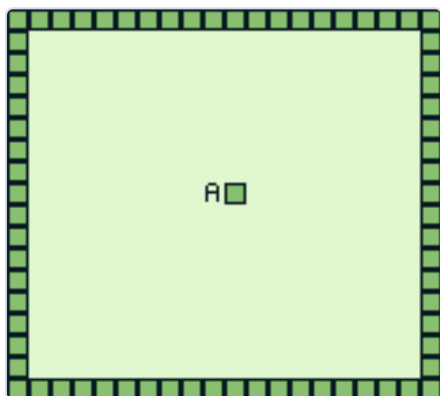
la toute première image — la console vient de commencer — image 1



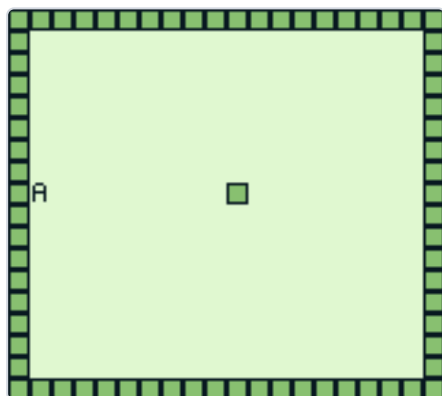
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

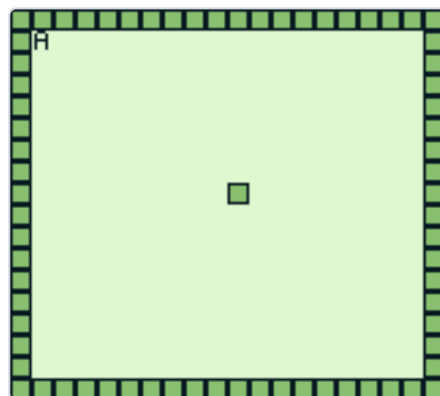
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



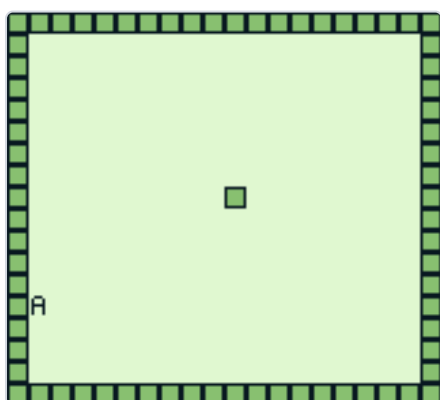
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Le curseur s'arrête net contre les murs et contre le bloc isolé.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d’être joués, et voici ce qu’ils ont mesuré.

- ✓ il s’arrête devant le bloc du milieu (colonne 9)
- ✓ il ne le traverse pas
- ✓ il s’arrête sous le mur du haut (ligne 1)
- ✓ et devant celui de gauche (colonne 1)

7. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C’est le même texte, écrit autrement.

MUR

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuancesLes quatre nuances de la console d’origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

MUR

tuile n° 44 — poser(x, y, MUR)

– zoom22×

+ zoom

Plein écran

Peindre ✓

Sélectionner et déplacer

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile

taille réelle — 8 × 8 pixels

Ensuite : **72. Ralentir sans compter les images à la main** — niveau 9 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++